

The Courier

VOL. 1, NO. 072

2026-03-30

FEATURES

Developer priorities throughout their career

Lea Verou

I made this chart in the amazing Excalidraw about two weeks ago:

It only took me 10 minutes! Shortly after, my laptop broke down into repeated kernel panics, and it spent about 10 days in service (I was in a remote place when it broke, so it took some time to get it to service). Yesterday, I was finally reunited with it, turned it on, launched Chrome, and saw it again. It gave me a smile, and I realized I never got to post it, so I tweeted this :

<https://twitter.com/LeaVerou/status/1306001020636540934>

The tweet kinda blew up! It seems many, many developers identify with it. A few also disagreed with it, especially with the “Does it actually work?” line. So I figured I should write a bit about the rationale behind it. I originally wrote it in a tweet , but then I realized I should probably post it in a less transient medium, that is more well suited to longer text.

When somebody starts coding, getting the code to work is already difficult enough, so there is no space for other priorities. Learning to formalize one’s thought to the degree a computer demands, and then serialize this thinking with an unforgiving syntax, is hard . Writing code that works is THE priority, and whether it’s good code is not even a consideration.

For more experienced programmers, whether it works is ephemeral: today it works, tomorrow a commit causes a regression, the day after another commit fixes it (yes, even with TDD. No testsuite gets close to 100% coverage). Whereas readability & maintainability do not fluctuate much. If they are not prioritized from the beginning, they are much harder to accomplish when you already have a large codebase full of technical debt.

Code written by experienced programmers that doesn’t work, can often be fixed with hours or days of debugging. A nontrivial codebase that is not readable can take months or years to rewrite. So one tends to gravitate towards prioritizing what is easier to fix.

The “peak of drought” and other over-abstractions

Many developers identified with the “peak of drought” . Indeed, like other aspects of maintainability, DRY is not even a concern at first. At some point, a programmer learns about the importance of DRY and gradually begins abstracting away duplication. However, you can have too much of a good thing: soon the need to abstract away any duplication becomes all consuming and leads to absurd, awkward abstractions which actually get in the way and produce needless couplings, often to avoid duplicating very little code, once. In my own “peak of drought” (which lasted far longer than the graph above suggests), I’ve written many useless functions, with parameters that make no sense, just to avoid duplicating a few lines of code once.

Many articles have been written about this phenomenon, so I’m not going to repeat their arguments here. As a programmer accumulates even more experience, they start seeing the downsides of over-abstraction and over-normalization and start favoring a more moderate approach which prioritizes readability over DRY when they are at odds.

A similar thing happens with design patterns too. At some point, a few years in, a developer reads a book or takes a course about design patterns. Soon thereafter, their code becomes so littered with design patterns that it is practically incomprehensible. “When all you have is a hammer, everything looks like a nail” . I have a feeling that Java and Java-like languages are particularly accommodating to this ailment, so this phenomenon tends to proliferate in their codebases. At some point, the developer has to go back to their past code, and they realize themselves that it is unreadable. Eventually, they learn to use design patterns when they are actually useful, and favor readability over design patterns when the two are at odds.

What aspects of your coding practice have changed over the years? How has your perspective shifted? What mistakes of the past did you eventually realize?

A nontrivial codebase that is not readable can take months or years to rewrite.

Lea Verou

What makes speakers happy

Lea Verou

I wish I could speak at CSSConf.eu , but unfortunately I had to decline the invitation, as it collided with a prior speaking engagement I had agreed on. I recently got another email from the organizers with an interesting question:

We want to make this event as stress-free for our speakers as possible. Since you spoke at a bunch of events, can you share a tip or two about what will make a speakers' life easier, and their stay more pleasant? Any typical mistakes we can avoid?

I thought it was lovely that they care about their speakers enough to ask this, this already places them above average. I started writing a reply, but I soon realized this is information that could be useful for other conference organizers as well, so I decided to post it here instead. So, what makes speakers happy?

The baseline

These are things every good conference is doing for their speakers, although they often miss one or two. They keep speakers happy, but they're not out of the ordinary.

Cover their flights, accommodation for the entire conference and ground transportation from/to the airport (with a car, not public transport!).

Do not expect them to go through the hassle of booking all those themselves and then sending you receipts. Offer it as an option, but book them yourself by default.

Do not book flights without confirming the itinerary and personal info with them first. Also, this sounds obvious, but it's surprising how many conferences have made this mistake with me: Type their name correctly when booking flights!

If hotel WiFi is not free, make sure it's covered and included in their reservation. Same goes for breakfast.

Offer a honorarium, at least to those who have to take time off work to speak at your event (e.g. freelancers). Even if your budget is small and can only give a tiny honorarium, it will at least cover their meals, cabs etc while there. If the honorarium is small and mainly intended to cover miscellaneous expenses of the trip, don't ask them to submit an invoice to claim it.

Have a speakers dinner before the event, where they can meet and socialize with the other speakers. This is also good for the conference, as they get the chance to catch up with their speaker friends (there aren't that many people on the conference circuit, so we often know each other and want to catch up) so they will talk more to the attendees during the conference. Make sure the speakers dinner does not overlap with the pre-party, if you have one.

Do a tech check before their talk to make sure everything is smooth. Have dongles for Mac laptops. Have clickers they could use. Use wireless lapel microphones. Have a reliable private wifi network for speakers to use if they need an internet connection for their talk.

Have breaks between talks so they have some margin of going overtime without impacting the schedule. If they are too stressed about going through their talk fast, it won't be a very good talk.

Going the extra mile

These are all things one or more conferences have done for me, but they are not generally common so they are a positive surprise when they happen, not something expected.

Book Business class flights, especially for longer flights where passengers are expected to sleep. It's so much more comfortable to sleep in a seat that fully reclines! I was incredibly grateful to the one conference that did this.

Cover incidentals in the hotel. Yes, it's a bit risky but come on, we're not rockstars. We won't screw you over. In most cases it will be a pretty small extra cost and it looks really good, it tells speakers you trust them and want them to have a good time.

Offer a speaker gift bag. It can contain all kinds of things: Stuff that will make their stay more comfortable (stain remover, travel toothbrush etc), souvenirs from the place since we rarely have time to do touristy stuff, alcohol for impromptu get togethers with other speakers, snacks to eat during a late night craving in the hotel room, anything goes and I've seen conferences put all kinds of stuff in there. It's a nice welcome gesture. Bonus points if they're personalized based on what you've researched about the speaker.

Send out a survey to the audience after the conference and let the speakers know how they did . Let them know what comments their talk got and how well they did compared to other speakers.

Also, make sure you read PPK's excellent Conference Organizer's Handbook .

When Things Go Wrong

Tony Wieczorek · Localytics Engineering

We believe the highest performing engineering teams have a process to identify, triage, fix and learn from service degradations and downtime. At Localytics, we build highly available and scalable systems, and part of the key to our success is learning from failures. One way we foster a learning culture is through how we respond to pages we get about our services. We wanted to share our process for constantly improving.

1. Notify, Coordinate and Fix in Slack: #prod-issue

At Localytics, we're big believers in the power of chatops to scale our team. We use integrations with Slack to deploy code, run database migrations, and monitor dozens of services.

Pingdom, New Relic, AWS Cloudwatch, and coded circuit breaker patterns post to a #prod-issue channel in Slack. All of those systems separately notify the team through PagerDuty integration. An engineer's first response to receiving a PagerDuty is to open Slack and check #prod-issue for more details, rapidly speeding up triaging the issue. While you might get a page about a particular service being down, checking #prod-issue lets you see if there are, for example, larger service problems before digging any further.

Since it's a standard Slack chat room, engineers can coordinate the response and debug the issue in realtime, no matter where they are physically located. The chat history is a convenient record of what happened during the event and serves as the official record of the incident.

2. Debug and Start Postmortem in Slack: #postmortem

After the initial crisis is over, our engineers move into the #postmortem Slack channel.

This is a place to hypothesize about what happened, challenge assumptions, and volunteer an Incident Leader to follow through. If the issue was a known risk that has now come to fruition, there's a good chance this conversation turns into a laundry list of ideas to fix the underlying issue. We often use the ideas generated here when we have the formal, blameless postmortem in Step 4.

3. Communicate to Company & Customers

A self-volunteered Incident Leader, usually the first person notified or "on the case", sends out an email to let the rest of the company know what happened. That distribution list goes to our Executive Team, Product leaders and Marketing department. It lets them decide whether to notify our customers proactively, or generate content to send to customers.

Here's the email template we use:

Subject: Issue Name, Date and Time

Hi everyone,

We experienced a production issue impacting [Service Name] . The issue began at [Time] on [Date] and ended on [Date] .

Issue team:

Engineering: @name

Product Management: @name

Product Marketing: @name

What is the issue and what caused it?

State what happened and why, in terms a general, non-technical audience can understand. Example: Database went down due to a bad query.

What is the impact to customers?

State the specific services that were impacted. Example: Push campaigns failed, attributions were not attributed, etc.

4.

Tony Wieczorek

Which customers were affected?

Include a link to a Google Sheet containing the following fields: org name, org_id , app_id , and campaign_id where applicable

What is the current status of the issue?

If the issue is resolved, state the fix and when it was deployed. If the issue is unresolved, give an approximate timeline for the fix.

Next steps

Product Marketing will follow up on this email thread with customer-facing messaging.

4. Hold a Blameless Postmortem

Within 24 business hours, the Incident Leader schedules and leads a Blameless Postmortem. This meeting can be anywhere from 30 minutes to several hours long. The point is to identify the processes, technologies and monitoring gaps that might have contributed to the issue.

The blameless concept is an important one. Postmortems should not be an exercise in identifying and disciplining specific engineers. Issues with services are rarely the fault of a single person. More likely, our internal processes and tools lead to the downtime, and the postmortem should focus on resolving those. This requires a belief that in a well-functioning team, people make rational decisions with the information they have at the time. It's not about blaming individuals.

Accessibility and Developers

Paul Irish

Jacob Thornton has an inspiring slide deck on accessibility that brings to fore some of the concerns that web developers have with implementing features around accessibility.

For a very long time, developers have been told about making websites ‘accessible’. Like it is a faucet that should be turned on or off. Till Victor Tsaran’s demonstration of using a screenreader (which was in 2007 - a very long time since accessibility evangelism began) - very few web developers had any idea of the real immediate impact of implementing accessibility features.

During a Nicholas Zakas & Victor Tsaran talk years ago I finally grokked the easiest rule for a first step towards accessibility. For such a long time, we conflated functionality while JavaScript-disabled with “being accessible”. It took me years to learn that making it keyboard-navigable was the top priority.

Just as a datapoint, The WebAIM survey results (published yesterday) reveal 98.6% of screenreader users have javascript enabled.

What we need

People like Jason Kiss and Steve Faulkner who detail the behavior and support in browsers and screenreaders are providing the developer community with invaluable knowledge, stuff that is far more worthwhile than publishing a yet another list of you-should-do-these-things. (See also my resources list in the semantics post).

Most developers have no idea what effect adding ARIA markup to our documents has on people using screenreaders. We need that. Actual before-and-after video stuff makes this topic real. See zomigi’s recent post: videos of screenreaders using ARIA , for great examples.

Most developers have no idea how to implement ARIA. The W3C Primer that looks like a spec is not at all appropriate for a web developer audience.

Just like it’s valuable to witness an a11y usability session where you see how disabled people use your site, I think it’d be useful for the accessibility community to see how developers build, because thus far there has been a disconnect in communicating effectively. I don’t think we need more on the ground evangelism yet. There is a dearth in online resources at the moment, developers need to know these practical ways of using and having users benefit from accessibility techniques:

Which features should I use and which should I avoid (which features are actively harmful because of shoddy screen reader/other a11y tech support?)

What are the top priorities for developers to implement?

Videos of screenreaders in use (Thanks Zoe!)

What one thing can I do as a web developer to my sites and apps that dramatically improves the UX for disabled users

How do some of these features work? What are they for? A demonstration of how each feature makes an impact or gets used by a user would be most helpful (e.g. how do focus rings work?).

Screenreaders & browsers

Also, I’m not quite sure how everyone else feels, but from here it feels like yelling at a fucking brick wall with JAWS and Window Eyes. We just kinda hope and pray their support for things improve. As a pragmatic approach, I tell developers that if it works in NVDA, good enough. We have a mess of browsers we need to support already, dealing with ATs that lag years behind is not at all feasible.

A lot of developers do not understand that accessibility first comes from implementing the platform accessibility API by browsers such that screen readers can take advantage of the rendering. Most operating systems expose an API that allows browsers to map what is rendered on the screen to one of the interfaces that the operating system’s accessibility API exposes.

Unfortunately, there has been no specification, so each browser has been left to do whatever it wants in terms of defining the relationship between a HTML element and the accessibility interface. Hopefully this will change with the HTML to Platform Accessibility API implementation guide and ensure each browser exposes each element consistently to the Platform Accessibility API.

Which shores up a greater concern: Better accessibility starts with the browser (and the platform). Browsers should be able to handle content loaded dynamically rather than expecting developers to do it for them. If content on a page that is displayed is altered, it needs to be noted to the accessibility api transparently instead of having developers bolt on extra markup to do so.

Accessibility should not be opt-in but opt-out.

This post is a elongated version of the comment I left on Karl Grove’s Barriers to improving the accessibility game plan . It was probably the best post on a11y I’ve ever read. I really liked his approach in evolving the a11y strategy and left my thoughts.

Thanks to Divya Manian for her thoughts and words on this post. Thanks to Jason Kiss for reviewing drafts.

June 1: Added link to freshly published WebAIM survey results 2012 . I encourage you to read the whole thing, as it’s a fascinating view into an audience we have challenges understanding.

Refactoring optional chaining into a large codebase: lessons learned

Lea Verou

Now that optional chaining is supported across the board , I decided to finally refactor Mavo to use it (yes, yes, we do provide a transpiled version as well for older browsers, settle down). This is a moment I have been waiting for a long time, as I think optional chaining is the single most substantial JS syntax improvement since arrow functions and template strings. Yes, I think it's more significant than async/await, just because of the mere frequency of code it improves. Property access is literally everywhere .

First off, what is optional chaining , in case you haven't heard of it before?

You know how you can't just do `foo.bar.baz()` without checking if `foo` exists, and then if `foo.bar` exists, and then if `foo.bar.baz` exists because you'll get an error? So you have to do something awkward like:

```
if (foo && foo.bar && foo.bar.baz) { foo.bar.baz(); }
```

Or even:

```
foo && foo.bar && foo.bar.baz && foo.bar.baz();
```

Some even contort object destructuring to help with this . With optional chaining, you can just do this:

```
foo?.bar?.baz?.()
```

It supports normal property access, brackets (`foo?.[bar]`), and even function invocation (`foo?.()`). Sweet, right??

Yes, mostly. Indeed, there is SO MUCH code that can be simplified with it, it's incredible. But there are a few caveats.

Patterns to search for

Suppose you decided to go ahead and refactor your code as well. What to look for?

There is of course the obvious `foo && foo.bar` that becomes `foo?.bar` .

There is also the conditional version of it, that we described in the beginning of this article, which uses `if()` for some or all of the checks in the chain.

There are also a few more patterns.

```
foo? foo.bar : defaultValue
```

Which can now be written as:

```
foo?.bar || defaultValue
```

or, using the other awesome new operator, the nullish coalescing operator :

```
foo?.bar ?? defaultValue
```

Array checking

which now becomes:

```
foo?.[2]
```

Note that this is no substitute for a real array check, like the one done by `Array.isArray(foo)` . Do not go about replacing proper array checking with duck typing because it's shorter. We stopped doing that over a decade ago .

Regex match

Forget about things like this:

```
let match = "#COFFEE".match(/#[A-Z]+/i); let hex = match && match[1];
```

Or even things like that:

```
let hex = ("#COFFEE".match(/#[A-Z]+/i) || [,])[1];
```

Now it's just:

```
let hex = "#COFFEE".match(/#[A-Z]+/i)?.[1];
```

In our case, I was able to even remove two utility functions and replace their invocations with this.

Feature detection

In simple cases, feature detection can be replaced by `?.` . For example:

```
if (element.prepend) element.prepend(otherElement);
```

becomes:

```
element.prepend?.(otherElement);
```

Don't overdo it

While it may be tempting to convert code like this:

```
if (foo) { something(foo.bar); somethingElse(foo.baz); andOneLastThing(foo.yolo); }
```

to this:

```
something(foo?.bar); somethingElse(foo?.baz); andOneLastThing(foo?.yolo);
```

Don't . You're essentially having the JS runtime check `foo` three times instead of one. You may argue these things don't matter much anymore performance-wise, but it's the same repetition for the human reading your code: they have to mentally process the check for `foo` three times instead of one. And if they need to add another statement using property access on `foo` , they need to add yet another check, instead of just using the conditional that's already there.

You still need to check before assignment

First 6 Months at Blinkit: Drishti Makhija

Jacob · Grofers / Blinkit Lambda

Image Credits: Kumkum Jain

Drishti (she/her/hers) works as a Software Developer Engineer (SDE) at Blinkit. She joined us about six months ago and is now a part of the tech team within the Product Listing and Search pod. She's always had the inclination to work at scale while making an impact and shipping a customer-first experience. In this post, Drishti recounts her journey with Blinkit and the lessons she learned along the way.

From Customer to Employee

This year on my friend's birthday, about 3 hours from midnight, I failed to get the ideal bouquet of flowers. Moreover, nobody was offering same-day flower delivery. Thanks to Instagram's recommendations, I found an advertisement for "Grofers is now Blinkit, delivering in minutes." I didn't think flowers belonged on this platform, but there's no harm in giving it a shot. To my delight, Blinkit offered a fantastic selection of flowers. That day, Blinkit gained another devoted customer. However, I wanted to learn what made the company tick and contribute to this fantastic product, not just be a customer.

Initial days at Blinkit

I always desired to work in a team with a direct line of sight to the end customer during my two years of experience in software; therefore, when given the option to select a vertical, I opted for Consumer. I joined the Product Listing (PLP) team as an SDE-1.

Coming from an MNC, I was astonished by the team structure here. Everyone here comes from a diverse range of disciplines, including design, data, product, and software. Yet, they all shared the same objective: "How can we serve the customer and the business by enhancing discovery?" My teammates' commitment and ownership inspired me to join the same boat immediately.

Things were going quickly initially, and it was a little daunting. However, my manager Mohit helped me adjust to this new environment. Rather than spoon-feeding me answers, he asked the right questions so I could figure out the best solution to any issue.

Sancheeta, as well as other team members from the Innovation Hub (IHUB), joined us a few days after I joined the team, and I collaborated with them on one of my first projects, "People also buy" and "Buy more, save more."

People also buy is a feature that recommends products bought along with the product added to the cart while Buy More Save more is for nudging the customers to buy more quantities of specific products and save extra. Throughout these projects, I gained knowledge about the power of data and how to leverage it to develop a consumer-focused product while attaining business goals. We approached these

initiatives with an experimental perspective, so if we fail, we fail soon—as opposed to waiting months for the complete feature to roll out. My manager, Sancheeta, has always held the conviction that if it's an experiment, we should swiftly deliver a smooth experience to the customers and let the data determine whether the idea is worthy of further iteration. In addition, even though it goes against our instinct as engineers, we should never over-engineer any problem.

After its initial incarnation, "People also buy" was discontinued due to poor results. At that point, among the most crucial teachings I learned as an engineer, was that failures shouldn't hold us back; instead, we should move on without dwelling on the setbacks. Contrarily, "Buy more, save more" was one of the projects I worked on up until this point, and it produced impressive results by raising the Average Order Value (the amount a customer spends per order).

Looking at the product via a new lens

Around my second month, one of the designers in our team invited us for a UX research interview with a customer. It was the first time I had the chance to hear the other end of the story. I realized that we work on problem statements day in and day out and look at the Blinkit app all the time, so sometimes we take the customer for granted—we feel that consumers will notice and use our feature as we do, but that is not always the case. This interview experience also taught me that I didn't want to be another engineer who just codes, but I want to be a Product Engineer who can ship an experience to the customer.

Over the following months, I did a couple of cross-team and business-critical projects. One such project was during the festive season. Due to some chaos, it dawned on me that I should start anything with "why" & "what" and then figure out the "how"—instead of directly working on a feature, figure out why I need to work on it.

There have been some ups and downs in these six months, but my learnings were exponential at every step. I came here to make an impact, and I did. My parents, who never trusted online shopping, now use Blinkit for every small and big thing they need. I, along with the PLP team, recently started working in collaboration with the Search team, and I am looking forward to exploring a new set of problem statements. I would like to thank Sancheeta, Mohit, Siddhant, Harsh, Himanshu, and the PLP & IHUB team for giving me such a rich experience.

Drishti is a SDE 1 at Blinkit. You can follow her on LinkedIn.

First 6 Months at Blinkit: Drishti Makhija was originally published in Lambda by Blinkit on Medium, where people are continuing the conversation by highlighting and responding to this story.

We approached these initiatives with an experimental perspective, so if we fail, we fail soon—as opposed to waiting months for the complete feature to roll out.

Jacob

Guide to Strings in Python

Dimitrije Stamenic · Stack Abuse

A string in Python is a sequence of characters . These characters can be letters, numbers, symbols, or whitespace, and they are enclosed within quotes. Python supports both single (' ') and double (" ") quotes to define a string, providing flexibility based on the coder's preference or specific requirements of the application.

More specifically, strings in Python are arrays of bytes representing Unicode characters.

Creating a string is pretty straightforward. You can assign a sequence of characters to a variable, and Python treats it as a string. For example:

```
my_string = "Hello, World!"
```

This creates a new string containing "Hello, World!". Once a string is created, you can access its elements using indexing (same as accessing elements of a list) and perform various operations like concatenation (joining two strings) and replication (repeating a string a certain number of times).

However, it's important to remember that strings in Python are immutable . This immutability means that once you create a string, you cannot change its content . Attempting to alter an individual character in a string will result in an error. While this might seem like a limitation at first, it has several benefits, including improved performance and reliability in Python applications. To modify a string, you would typically create a new string based on modifications of the original.

Python provides a wealth of methods to work with strings , making string manipulation one of the language's strong suits. These built-in methods allow you to perform common tasks like changing the case of a string, stripping whitespace, checking for substrings, and much more, all with simple, easy-to-understand syntax, which we'll discuss later in this article.

As you dive deeper into Python, you'll encounter more advanced string techniques. These include formatting strings for output, working with substrings , and handling special characters. Python's string formatting capabilities, especially with the introduction of f-Strings in Python 3.6, allow for cleaner and more readable code. Substring operations, including slicing and finding, are essential for text analysis and manipulation.

Moreover, strings play nicely with other data types in Python, such as lists. You can convert a string into a list of characters, split a string based on a specific delimiter, or join a collection of strings into a single string. These operations are particularly useful when dealing with data input and output or when parsing text files.

In this article, we'll explore these aspects of strings in Python, providing practical examples to illustrate how to

effectively work with strings. By the end, you'll have a solid foundation in string manipulation, setting you up for more advanced Python programming tasks.

Strings are one of the most commonly used data types in Python, employed in diverse scenarios from user input processing to data manipulation. This section will explore the fundamental operations you can perform with strings in Python.

In Python, you can create strings by enclosing a sequence of characters within single, double, or even triple quotes (for multiline strings). For example, `simple_string = 'Hello'` and `another_string = "World"` are both valid string declarations. Triple quotes, using `'''` or `"""` , allow strings to span multiple lines, which is particularly useful for complex strings or documentation.

The simplest way to create a string in Python is by enclosing characters in single (') or double (") quotes .

Note: Python treats single and double quotes identically

This method is straightforward and is commonly used for creating short, uncomplicated strings:

```
# Using single quotes greeting = 'Hello, world!'
```

```
# Using double quotes title = "Python Programming"
```

For strings that span multiple lines , triple quotes (`'''` or `"""`) are the perfect tool. They allow the string to extend over several lines, preserving line breaks and white spaces:

```
# Using triple quotes multi_line_string = """This is a multi-line string in Python."""
```

Sometimes, you might need to include special characters in your strings , like newlines (`\n`), tabs (`\t`), or even a quote character. This is where escape characters come into play, allowing you to include these special characters in your strings:

```
# String with escape characters escaped_string = "He said, \nPython is amazing!"
```

Printing the `escaped_string` will give you:

```
He said, "Python is amazing!" And I couldn't agree more.
```

Accessing and Indexing Strings

Once a string is created, Python allows you to access its individual characters using indexing. Each character in a string has an index, starting from 0 for the first character.

For instance, in the string `s = "Python"` , the character at index 0 is 'P'. Python also supports negative indexing, where -1 refers to the last character, -2 to the second-last, and so on. This feature makes it easy to access the string from the end.

Subverting .NET Type Safety with 'System.Runtime.CompilerServices.Unsafe'

Matt Warren (.NET)

In which we use System. Runtime. CompilerServices. Unsafe a generic API ("type-safe" but still "unsafe") and mess with the C# Type System!

The post covers the following topics:

What it is and why it's useful

How it works

Code samples

Tricks you can do with it

Using it safely

What it is and why it's useful

The XML documentation comments for System. Runtime. CompilerServices. Unsafe state that it:

Contains generic, low-level functionality for manipulating pointers.

But we can get a better understanding of what it is by looking at the actual API definition from the current NuGet package (4.0.0) :

```
// Contains generic, low-level functionality for manipulating pointers. public static class Unsafe { // Casts the given object to the specified type. public static T As ( object o ) where T : class
```

```
// Returns a pointer to the given by-ref parameter. public static void * AsPointer ( ref T value );
```

```
// Copies a value of type T to the given location. public static void Copy ( void * destination , ref T source );
```

```
// Copies a value of type T to the given location. public static void Copy ( ref T destination , void * source );
```

```
// Copies bytes from the source address to the destination address. public static void CopyBlock ( void * destination , void * source , uint byteCount );
```

```
// Initializes a block of memory at the given location with a given initial value. public static void InitBlock ( void * startAddress , byte value , uint byteCount );
```

```
// Reads a value of type T from the given location. public static T Read ( void * source );
```

```
// Returns the size of an object of the given type parameter. public static int SizeOf ();
```

```
// Writes a value of type T to the given location. public static void Write ( void * destination , T value ); }
```

Note: I edited the the XML doc-comments for brevity, the full versions are available in the source . There are also some additional methods that have been added to the API , but to make use of them you have to use a version of the C# compiler with support for ref returns and locals .

However this doesn't really tell us why it's useful , to get some background on that we can look at the GitHub issue "Provide a generic API to read from and write to a pointer" :

So at a high-level the goals of the System. Runtime. CompilerServices. Unsafe library are to:

Provide a safer way of writing low-level unsafe code

Without this library you have to resort to fixed and pointer manipulation, which can be error prone

Allow access to functionality that can't be expressed in C#, but is possible in IL

For instance Unsafe. Sizeof () allows access to the Sizeof IL Opcode

Save developers from having to repeatedly write the same unsafe code

There are already code-bases making use of it , including the Kestrel the high-performance web server, based on libuv.

It's also worth pointing out that the library is primarily for use with a Value Type (int, float, etc) rather than a class or Reference type. You can use it with classes, however you have to pin them first , so they don't move about in memory whilst you are working with the pointer.

Update: It was pointed out to me that Niels wrote an initial implementation of this library in a separate project , before Microsoft made their own version.

How it works

Because the library allows access to functionality that can't be expressed in C#, it has to be written in raw IL , which is then compiled by a custom build-step. As an example we will look at the AsPointer method, which has the following signature:

```
public static void * AsPointer ( ref T value )
```

The IL for this is shown below, note how the ref keyword becomes & in IL and is expressed as !! T :

```
.method public hidebysig static void* AsPointer (!! T& 'value') cil managed aggressiveinlining { .custom instance void System. Runtime. Versioning. NonVersionableAttribute::ctor() = ( 01 00 00 00 ) .maxstack 1 ldarg.0 conv.u ret } // end of method Unsafe:: AsPointer
```

Here we can see that it's making use of the conv.u IL instruction. For reference the explanation of this, along with some

The Skinny on IE's Update Policy

Paul Irish

Every month, WebKit, Firefox and Opera are shipping incredible features; IE10 is also going to settle up and even out those HTML5 Test scores (plus some features they may debut, like Grid Layout!). But while these features are becoming available in some browsers, most of us can't use them because we have a sizable audience who have been left behind on old browsers. A while ago I made a big fuss about IE's lack of solid upgrade path and how it meant we'd end up with 10 major versions of IE in the wild. But luckily, things took a turn for the better when Microsoft announced a new autoupdate policy for IE.

I wanted to take a moment to explain what this IE update policy means for us web developers.

The big change is how IE now interacts with Windows Update: Before you had to opt-in to have "Recommended" updates installed automatically. It's likely few folks did. Now, IE upgrades are shipping as "Important" and this class of updates are defaulted to install automatically. A user can still opt-out and immediately after the announcement of this feature they also published details on how enterprises can opt-out. But still, this is a good move, it helps.

The policy manifests as:

Windows XP holdouts on IE6 or IE7 get the boot up to IE8

Windows Vista and 7 users still on IE7 or even on IE8 get shunted up to IE9

IE10 only works on Win7+, and while no policy has been published for Win7 users, it's likely any IE8/IE9 users will be moved up to IE10.

A very smart policy call was that this update policy also includes all of China, which is important because most of their Windows installs are not genuine. Microsoft decided they should get the IE bump anyway, which is great because China has been dominated by IE6 and IE8 for ages.

This update procedure was scheduled to start in Australia and Brazil in January 2012. If we look at what happened in Brazil, we have a good shift of users but it's flattened out with a nontrivial amount of left behind users: (see the two blue lines in the chart below)

(Source: StatCounter Global Stats - Browser Version (Partially Combined) Market Share)

Australia has a similar swap of users from IE8 to IE9 but IE8 still remains steady above 10% overall share there.

Microsoft is in a tough spot in that they have enterprise customers who have developed their intranet applications in an extremely poor manner and they break outside of old browsers. But we're going to need some more pushes from them and us, the developer community, to get in a better

state. We want to develop for the web platform of now, not the platform of four years ago.

TL;DR: There is now a policy in place, but evidence indicates that it's not as effective in eradicating these zombie browsers as what we need. Personal opinion: we need to do better.

Hopefully this clarifies a bit about the mechanics of the update procedure. Please do correct me if anything is wrong and I'll update the post.

2012.08.07: Added TLDR.

2012.07.02: To see this update policy in better context it's probably useful to Read this Developer's Guide to browser adoption rates and check the charts from Ars Technica below.

2012.09.21: Months later, the net effect of the update campaign is complete. It pushed a whole bunch of users up to IE9 but IE8 usage remains solid. :(WinXP is a big problem here, obviously. We have to explore other ways to move these users off a zombie browser...

(Consider the two blue lines)

Source: StatCounter Global Stats - Browser Version (Partially Combined) Market Share

Research based on the .NET Runtime

Matt Warren (.NET)

Over the last few years, I've come across more and more research papers based, in some way, on the 'Common Language Runtime' (CLR).

Using the .NET Runtime as a case-study

to prove its correctness, study how it works or analyse its behaviour

“ It was formed in 1991, with the intent to advance state-of-the-art computing and solve difficult world problems through technological innovation in collaboration with academic, government, and industry researchers ” (according to Wikipedia)

Papers based on the Mono Runtime

a ' Cross-Platform, open-source .NET framework '

Using 'Rotor', real name 'Shared Source CLI (SSCLI)'

from Wikipedia “ Microsoft provides the Shared Source CLI as a reference CLI implementation suitable for educational use ”

Any papers I've missed? If so, please let me know in the comments or on Twitter

.NET Runtime as a Case-Study

Pitfalls of C# Generics and Their Solution Using Concepts (Belyakova & Mikhalkovich, 2015)

Efficient Compilation of .NET Programs for Embedded Systems (Sallenaveab & Ducournaub, 2011)

Type safety of C# and .Net CLR (Fruja, 2007)

Modeling the .NET CLR Exception Handling Mechanism for a Mathematical Analysis (Fruja & Börger, 2006)

Analysis of the .NET CLR Exception Handling Mechanism (Fruja & Börger, 2005)

A Modular Design for the Common Language Runtime (CLR) Architecture (Fruja, 2005)

Cross-language Program Slicing in the .NET Framework (Pócza, Biczó & Porkoláb, 2005)

Design and Implementation of a high-level multi-language .NET Debugger (Strein, 2005)

A High-Level Modular Definition of the Semantics of C# (Börger, Fruja, Gervasi & Stärk, 2004)

An ASM Specification of C# Threads and the .NET Memory Model (Stärk and Börger, 2004)

Common Language Runtime : a new virtual machine (Ferreira, 2004)

JVM versus CLR: a comparative study (Singer, 2003)

Runtime Code Generation with JVM And CLR (Sestoft, 2002)

Project Snowflake: Non-blocking safe manual memory management in .NET (Parkinson, Vaswani, Costa, Deligianis, Blankstein, McDermott, Balkind & Vytiniotis, 2017)

Simple, Fast and Safe Manual Memory Management (Kedia, Costa, Vytiniotis, Parkinson, Vaswani & Blankstein, 2017)

Uniqueness and Reference Immutability for Safe Parallelism (Gordon, Parkinson, Parsons, Bromfield & Duffy, 2012)

A study of concurrent real-time garbage collectors (Pizlo, Petrank & Steensgaard, 2008)

Optimizing concurrency levels in the .net threadpool: A case study of controller design and implementation (Hellerstein, Morrison & Eilebrecht, 2008)

Stopless: a real-time garbage collector for multiprocessors. (Pizlo, Frampton, Petrank & Steensgaard, 2007)

Securing the .NET Programming Model (Kennedy, 2006)

Combining Generics, Pre-compilation and Sharing Between Software-Based Processes (Syme & Kennedy, 2004)

Formalization of Generics for the .NET Common Language Runtime (Yu, Kennedy & Syme, 2004)

Runtime Verification of .NET Contracts (Barnett & Schulte, 2003)

Design and Implementation of Generics for the .NET Common Language Runtime (Kennedy & Syme, 2001)

Typing a Multi-Language Intermediate Code (Gordon & Syme, 2001)

Static and Dynamic Analysis of Android Malware and Goodware Written with Unity Framework (Shim, Lim, Cho, Han & Park, 2018)

Reducing startup time of a deterministic virtualizing runtime environment (Däumler & Werner, 2013)

Detecting Clones Across Microsoft .NET Programming Languages (Al-Omari, Keivanloo, Roy & Rilling, 2012)

Language-independent sandboxing of just-in-time compilation and self-modifying code (Ansel & Marchenko, 2012)

VMKit: a Substrate for Managed Runtime Environments (Geoffray, Thomas, Lawall, Muller & Folliot, 2010)

MMC the Mono Model Checker (Ruys & Aan de Brugh, 2007)

How the dotnet CLI tooling runs your code

Matt Warren (.NET)

Just over a week ago the official 1.0 release of .NET Core was announced, the release includes:

the .NET Core runtime, libraries and tools and the ASP.NET Core libraries.

However alongside a completely new, revamped, xplat version of the .NET runtime, the development experience has been changed, with the dotnet based tooling now available (Note : the tooling itself is currently still in preview and it's expected to be RTM later this year)

So you can now write:

```
dotnet new dotnet restore dotnet run
```

and at the end you'll get the following output:

```
Hello World!
```

It's the dotnet CLI (Command Line Interface) tooling that is the focus of this post and more specifically how it actually runs your code, although if you want a tl;dr version see this tweet from @citizenmatt :

Traditional way of running .NET executables

As a brief reminder, .NET executables can't be run directly (they're just IL, not machine code), therefore the Windows OS has always needed to do a few tricks to execute them, from CLR via C# :

After Windows has examined the EXE file's header to determine whether to create a 32-bit process, a 64-bit process, or a WoW64 process, Windows loads the x86, x64, or IA64 version of MSCorEE.dll into the process's address space. ... Then, the process' primary thread calls a method defined inside MSCorEE.dll. This method initializes the CLR, loads the EXE assembly, and then calls its entry point method (Main). At this point, the managed application is up and running.

New way of running .NET executables

```
dotnet run
```

So how do things work now that we have the new CoreCLR and the CLI tooling? Firstly to understand what is going on under-the-hood, we need to set a few environment variables (COREHOST_TRACE and DOTNET_CLI_CAPTURE_TIMING) so that we get a more verbose output:

Here, amongst all the pretty ASCII-art, we can see that dotnet run actually executes the following cmd:

```
dotnet -additionalprobingpath C:\Users\matt\.nuget\packages c:\dotnet\bin\Debug\netcoreapp1.0\myapp.dll
```

Note : this is what happens when running a Console Application. The CLI tooling supports other scenarios, such as self-hosted web sites, which work differently.

dotnet exec and corehost

Up-to this point everything was happening within managed code, however once dotnet exec is called we jump over to unmanaged code within the corehost application. In addition several other .dlls are loaded, the last of which is the CoreCLR runtime itself (click to go to the main source file for each module):

hostpolicy.dll

hostfxr.dll

coreclr.dll

The main task that the corehost application performs is to calculate and locate all the dlls needed to run the application, along with their dependencies. The full output is available, but in summary it processes:

99 Managed dlls ("Adding runtime asset..")

136 Native dlls ("Adding native asset..")

There are so many individual files because the CoreCLR operates on a "pay-for-play" model, from Motivation Behind .NET Core :

By factoring the CoreFX libraries and allowing individual applications to pull in only those parts of CoreFX they require (a so-called "pay-for-play" model), server-based applications built with ASP.NET 5 can minimize their dependencies.

Finally, once all the housekeeping is done control is handed off to corehost, but not before the following properties are set to control the execution of the CoreCLR itself:

TRUSTED_PLATFORM_ASSEMBLIES =

Paths to 235 .dlls (99 managed, 136 native), from C:\Program Files\dotnet\shared\Microsoft.NETCore.App\1.0.0-rc2-3002702

APP_PATHS =

c:\dotnet\bin\Debug\netcoreapp1.0

APP_NI_PATHS =

c:\dotnet\bin\Debug\netcoreapp1.0

NATIVE_DLL_SEARCH_DIRECTORIES =

C:\Program Files\dotnet\shared\Microsoft.NETCore.App\1.0.0-rc2-3002702

c:\dotnet\bin\Debug\netcoreapp1.0

Simplify Regular Expressions with RegExpBuilderJS

Scott Robinson · Stack Abuse

Regular expressions are one of the most powerful tools in a developer's toolkit. But let's be honest, regex kind of sucks to write. Not only is it hard to write, but it's also hard to read and debug too. So how can we make it easier to use?

In its traditional form, regex defines powerful string patterns in a very compact statement. One trade-off we can make is to use a more verbose syntax that is easier to read and write. This is the purpose of a package like `regexp-builderjs`.

The `regexpbuilderjs` package is actually a port of the popular PHP package, `regexpbuilderphp`. The `regexpbuilderphp` package itself is a port of an old JS package, `regexpbuilder`, which now seems to be gone. This new package is meant to continue the work of the original `regexpbuilder` package.

All credit goes to Andrew Jones for creating the original JS version and Max Girkens for the PHP port.

To install the package, you can use `npm`:

```
$ npm install regexpbuilderjs
```

Here's a simple example of how you can use the package:

```
const RegExpBuilder = require('regexpbuilderjs');  
  
const builder = new RegExpBuilder();  
const regex = builder.startOfLine().exactly(1).of('S').getRegExp();
```

Now let's break this down a bit. The `RegExpBuilder` class is the main class that you'll be using to build your regular expressions. You can start by creating a new instance of this class and chain methods together to create your regex:

`startOfLine()` : This method adds the `^` character to the regex, which matches the start of a line.

`exactly(1)` : This method adds the `{1}` quantifier to the regex, which matches exactly one occurrence of a given character or group.

`of('S')` : This method adds the `S` character to the regex.

`getRegExp()` : This method returns the final `RegExp` object that you can use to match strings.

With this, you can match strings like "Scott", "Soccer", or "S418401".

This is great and all, but this is probably a regex string you could come up with on your own and not struggle too much to read. So now let's see a more complex example:

```
const builder = new RegExpBuilder();  
  
const regex = builder.startOfInput().exactly(4).digits().then('_').exactly(2).dig-
```

```
its().then('_').min(3).max(10).letters().then('.').anyOf(['png', 'jpg', 'gif']).endOfInput().getRegExp();
```

This regex is meant to match filenames, which may look like:

2020_10_hund.jpg

2030_11_katze.png

4000_99_maus.gif

Some interesting parts of this regex is that we can specify type of strings (i.e. `digits()`), min and max occurrences of a character or group (i.e. `min(3).max(10)`), and a list of possible values (i.e. `anyOf(['png', 'jpg', 'gif'])`).

For a full list of methods you can use to build your regex, you can check out the [documentation](#).

This is just a small taste of what you can do with `regexp-builderjs`. The package is very powerful and can help you build complex regular expressions in a more readable and maintainable way.

Comments, questions, and suggestions are always welcome! If you have any feedback on how this could work better, feel free to reach out on [X](#). In the meantime, you can check out the repo on [GitHub](#) and give it a star while you're at it.

Agent Psychosis: Are We Going Insane?

Armin Ronacher · Armin Ronacher (Lucumr)

You can use Polecats without the Refinery and even without the Witness or Deacon. Just tell the Mayor to shut down the rig and sling work to the polecats with the message that they are to merge to main directly. Or the polecats can submit MRs and then the Mayor can merge them manually. It's really up to you. The Refineries are useful if you have done a LOT of up-front specification work, and you have huge piles of Beads to churn through with long convoys.

— Gas Town Emergency User Manual , Steve Yegge

Many of us got hit by the agent coding addiction. It feels good, we barely sleep, we build amazing things. Every once in a while that interaction involves other humans, and all of a sudden we get a reality check that maybe we overdid it. The most obvious example of this is the massive degradation of quality of issue reports and pull requests. As a maintainer many PRs now look like an insult to one's time, but when one pushes back, the other person does not see what they did wrong. They thought they helped and contributed and get agitated when you close it down.

But it's way worse than that. I see people develop parasocial relationships with their AIs, get heavily addicted to it, and create communities where people reinforce highly unhealthy behavior. How did we get here and what does it do to us?

I will preface this post by saying that I don't want to call anyone out in particular, and I think I sometimes feel tendencies that I see as negative, in myself as well. I too, have thrown some vibes up to other people's repositories.

Our Little Dæmons

In His Dark Materials, every human has a dæmon, a companion that is an

externally visible manifestation of their soul. It lives alongside as an

animal, but it talks, thinks and acts independently. I'm starting to relate our

relationship with agents that have memory to those little creatures. We become

dependent on them, and separation from them is painful and takes away from our

new-found identity. We're relying on these little companions to validate us and

to collaborate with. But it's not a genuine collaboration like between humans,

it's one that is completely driven by us, and the AI is just there for the ride.

We can trick it to reinforce our ideas and impulses. And we act through this

AI. Some people who have not programmed before, now wield tremendous powers,

but all those powers are gone when their subscription hits a rate limit and

their little dæmon goes to sleep.

Then, when we throw up a PR or issue to someone else, that contribution is the

result of this pseudo-collaboration with the machine. When I see an AI pull

request come in, or on another repository, I cannot tell how someone created it,

but I can usually after a while tell when it was prompted in a way that is

fundamentally different from how I do it. Yet it takes me minutes to figure

this out. I have seen some coding sessions from others and it's often done with

clarity, but using slang that someone has come up with and most of all: by

completely forcing the AI down a path without any real critical thinking.

Particularly when you're not familiar with how the systems are supposed to work,

giving in to what the machine says and then thinking one understands what is

going on creates some really bizarre outcomes at times.

But people create these weird relationships with their AI agent and once you see how some prompt their machines, you realize that it dramatically alters what comes out of it. To get good results you need to provide context, you need to make the tradeoffs, you need to use your knowledge. It's not just a question of using the context badly, it's also the way in which people interact with the machine. Sometimes it's unclear instructions, sometimes it's weird role-playing and slang, sometimes it's just swearing and forcing the machine, sometimes it's a weird ritualistic behavior. Some people just really ram the agent straight towards the most narrow of all paths towards a badly defined goal with little concern about the health of the codebase.

Addicted to Prompts

These dæmon relationships change not just how we work, but what we produce. You can completely give in and let the little dæmon run circles around you. You can reinforce it to

CSS3 colors, today (MediaCampAthens session)

Lea Verou

Yesterday, I had a session at MediaCampAthens (a BarCamp-style event), regarding CSS3 colors. If you've followed my earlier posts tagged with "colors", my presentation was mostly a sum-up of these.

It was my first presentation ever, actually, the first time I talked to an audience for more than 1 minute : P . This caused some goofs:

When introducing myself, I said completely different things than I intended to and ended up sounding like an arrogant moron : P

I tried not to look at the audience too much, in order to avoid sounding nervous, and this caused me to completely ignore 2 questions (as I found out afterwards)! How embarrassing!

At a certain point, I said "URL" instead of "domain" : P

Also, I had prepared some screenshots (you'll see them in the ppt) and the projector completely screwed them up, as it showed any dark color as black.

Apart from those, I think it went very well, I received lots of positive feedback about it and the audience was paying attention, so I guess they found it interesting (something that I didn't expect : P).

Here is the presentation:

Please note that Slideshare messed up slide #8 and the background seems semi-

transparent grey instead of semi-transparent white.

By the way, I also thought afterwards that I had made a mistake: -ms-filter is not required if we combine the gradient filter with Data URIs, since IE8 supports Data URIs (for images at least). Oops, I hate making mistakes that I can't correct.

Here are some photos from my session . If I did it correctly, every facebook user can see them. If I messed things up, tell me : P

I'm going to MIT!!

Lea Verou

Last year, I did something crazy, that I've been wanting to do since I was little: I applied to MIT's PhD program in Electrical Engineering and Computer Science.

It was not only crazy because I have been working for several years already, but also because I only applied to MIT, as I decided I did not want to go to any other university, both for pragmatic and emotional reasons. As any prospective grad student will tell you, applying to only one top university is recipe for failure. I didn't tell many people, but everyone who knew thought I'd get in — except me. You see, I wasn't a typical candidate. Sure, I have done lots of things I'm proud of, but I didn't have an amazing GPA or publications in prestigious academic conferences.

It felt like a very long shot, so you can imagine my excitement when I received the letters of acceptance, about a week ago. I will remember that moment forever. I was watching Breaking Bad, feeling miserable over a breakup that happened only a few hours earlier. About a minute into the episode (s05e09), I saw an email notification titled "Your application to MIT EECS". My first thought was that there was some problem with my application. And then I read the first few lines:

Dear Michailia Verou:

If you have not already heard from them, you will shortly receive a letter from the EECS department at

MIT, informing you that you have been admitted to the graduate program in Computer Science at MIT next fall. Congratulations!!

WHAAAA? Was it a scam? But then, how did they have all my details? Holy mother of the Flying Spaghetti Monster, I got in!!! Soon thereafter, a letter from CSAIL followed (where I said I wanted to work, specifically in the UID), and then even more letters. I started calling everyone who knew I applied to share the news, though it proved quite hard to form sentences instead of uncontrollably screaming in joy. I was (and am!) so excited about the future, that it completely overshadows any other life problems (at least for the time being).

Of course, my happiness is mixed with sheer terror. I keep worrying that I will be the dumbest person in the room, or that I don't remember as much from my undergrad studies as the others will. I'm even terrified of meeting my future advisor(s) in case getting to know me better makes them wonder why I was accepted. But I try to remind myself about impostor syndrome , and from what I've read in forums & blogs, it seems that I'm not alone in having such fears.

I held off blogging about it until I felt I was able to write something coherent, but I can't wait to share my excitement any longer.

To the future!

To real life plot twists!

To MIT!

Boy, I'm thrilled. : D

Scaling HAProxy on AKS for Billions of Transactions with Dynamic Autoscaling and Token Management

Shoeb Khan · Haptik Tech

At our scale, we needed to handle billions of transactions efficiently - routing across multiple environments and internal data centers - while maintaining reliability, security, and dynamic control. To achieve this, we chose HAProxy, a proven, high-performance load balancer known for its lightweight footprint, flexibility, and ability to handle massive concurrency with minimal overhead.

This blog walks you through how we deployed HAProxy in high-availability (HA) mode on Azure Kubernetes Service (AKS) -complete with autoscaling powered by KEDA, seamless integration with Azure Application Gateway, and dynamic token management using Azure File Share , all designed to deliver scalable, self-hosted traffic routing at enterprise scale.

ALSO READ: How We Took Our Kubernetes Autoscaling from Basic to Advanced Mode with Istio Metrics

ANALYSIS

Advanced Performance Audits with Dev-Tools

Paul Irish

Recently, I've spent some time recently profiling real-world mobile websites. Using the 1000/100/6 performance model 1 , and spelunking deep into each app, the findings have been fascinating.

I've written up case study documents for each, incorporating all the findings:

Illustrated diagnoses for the poor performance

What actions the developer should take

How Chrome's tooling should improve

Questions and insights for the rendering engine (Blink)

→ Perf Audits: CNet, Time, Google Play

In this doc, we look at the scrolling of CNET, input latency on CNET, some very interesting challenges on the responsive Time.com, and infinite scroll on Google Play's desktop site.

The intended audience is browser engineers and performance-minded frontend developers. It's fairly advanced, but I'm spelunking deep to identify how the sites butt heads with the browser APIs and architecture.

Lastly, we're using this research to improve Chrome DevTools and what you hear from Chrome.

Wikipedia eng team scrutinizing their performance millisecond by millisecond.

(Yes, it's a long paper printout of the Chrome DevTools timeline flamechart :)

(BTW, use this link to view the same doc but with comments enabled)

1 - More on this performance model later. Stay tuned.

StronglyTyped: A library for strongly typed properties & constants in JavaScript

Lea Verou

I'll start by saying I love the loosely typed nature of JavaScript. When I had to work with strongly typed languages like Java, it always seemed like an unnecessary hassle. On the contrary, my boyfriend even though very proficient with HTML, CSS and SVG, comes from a strong Java background and hates loosely typed scripting languages. So, to tempt him into JS and keep him away from heavy abstractions like Objective-J, I wrote a little library that allows you to specify strongly typed properties (and since global variables are also properties of the window object, those as well) of various types (real JS types like Boolean, Number, String etc or even made up ones like Integer) and constants (final properties in Java). It uses ES5 getters and setters to do that and falls back to regular, loosely typed properties in non-supporting browsers.

Also, as a bonus, you get cross-browser `Function.prototype.bind` and `Array.prototype.forEach` and a robust type checking function: `StronglyTyped.is(type, value)`.

Example: Strongly typed properties

You define strongly typed properties by using the corresponding methods of the `StronglyTyped` object. For example, the following snippet defines a boolean property called "foo" on an object literal:

```
var o = {};  
  
StronglyTyped.boolean(o, 'foo', true);  
  
console.log(o.foo); // prints true  
  
o.foo = false; console.log(o.foo); // prints false  
  
o.foo = 'bar'; // TypeError: foo must be of type Boolean. bar is not.
```

Example: Constants

You define constants by using the `constant` method of the `StronglyTyped` object. For example, the following snippet defines a global `MAGIC_NUMBER` constant:

```
var o = {};  
  
StronglyTyped.constant(window, 'MAGIC_NUMBER',  
3.1415926535);  
  
console.log(MAGIC_NUMBER); // prints 3.1415926535  
  
MAGIC_NUMBER = 4; console.log(MAGIC_NUMBER); //  
prints 3.1415926535
```

Please note that constants only become read-only after they first get a non-undefined value. For example:

```
StronglyTyped.constant(window, 'MAGIC_NUMBER');
```

```
console.log(MAGIC_NUMBER); // prints undefined
```

```
MAGIC_NUMBER = undefined;
```

```
console.log(MAGIC_NUMBER); // prints undefined
```

```
MAGIC_NUMBER = 3.1415926535;  
console.log(MAGIC_NUMBER); // prints 3.1415926535
```

```
MAGIC_NUMBER = 4; console.log(MAGIC_NUMBER); //  
prints 3.1415926535
```

Supported types

RegExp

null and undefined are valid in every type. NaN and Infinity values are accepted in both the Number and the Integer types.

If you want to use a type that's not among the above but either is native to the browser (for example `Element`) or a global object, you can use the generic method `StronglyTyped.property(type, object, property [, initialValue])`:

```
var o = {};
```

```
StronglyTyped.property('Element', o, 'foo',  
document.body);
```

```
console.log(o.foo); // prints a representation of the element
```

```
o.foo = document.head; console.log(o.foo); // prints a rep-  
resentation of the element
```

```
o.foo = 5; // TypeError: foo must be of type Element. 5 is not.
```

Browser support

It should work on every browser that supports `Object.defineProperty` or `__defineGetter__` and `__defineSetter__`. As you can see from kangax's awesome compatibility tables for `Object.defineProperty` and `__defineGetter__`, those are:

Firefox 3.5+

IE8 (only on DOM elements)

IE9+

Opera 10.5+

Chrome 5+

Safari 4+

Konqueror 4.4+

However, it's only verified to work in:

foo); / prints a representation of the element o.

Lea Verou

State of HTML 2025 now open!

Lea Verou

tl;dr : State of HTML 2025 survey is now open! Take it now

Mamma mia, here we go again!

About two weeks ago, I announced that I was back leading this year's State of HTML 2025 survey, after a one year hiatus. We are grateful for all the suggestions that poured in, they were immensely helpful in shaping the survey. After two weeks of hard work from a small team spanning three continents, we are finally ready to launch!

I would urge each and every one of you that works with the web platform to fill out this survey . It's a unique opportunity to have your voice heard in the browser vendors' decision-making process. Survey results are used by browsers to prioritize roadmaps — the reason Google is funding this. The results from State of ... surveys directly feed into prioritization for next year's Interop project.

Time spent thoughtfully filling them out is an investment that can come back to you tenfold in the form of seeing features you care about implemented, browser incompatibilities being prioritized, and gaps in the platform being addressed. In addition to browsers, several standards groups are also using the results for prioritization and decision-making.

Additionally, you get to learn about new and upcoming features you may have missed, and get a personalized, sharable score at the end to see how you compare to other respondents!

Take State of HTML 2025 Survey

While the survey will be open for about a month, responses entered within the first two weeks (until end of July) will have a much higher impact on the Web, as preliminary data will be directly used to inform Interop 2026.

I would like to thank the following people for their help in fleshing out the survey:

Sacha Greif for actually implementing the survey and tirelessly discussing my ideas

My apprentice Dmitry Sharabin for various fixes and improvements

My Google contacts, Philip Jägenstedt and Kadir Topal for making my involvement possible

And everyone who responded to my earlier call for suggestions — these were invaluable in shaping the survey, and I wish I could include them all!

FAQ

What's new this year?

We spent a lot of time thinking about which features we are asking about and why. As a result, we added 35 new features, and removed 18 existing ones to make room. This is probably one of the hardest parts of the process, as we had to make some tough decisions.

We are also using the Web Components section to pilot a new format for pain points questions, consisting of a multiple choice question with common pain points, followed by the usual free form text list:

While this increases the number of questions, we are hoping it will reduce survey fatigue by allowing participants to skip the freeform question more frequently (or spend less time on it) if most of their pain points have already been covered by the multiple choice question.

Last but not least, we introduced browser support icons for each feature, per popular request:

Can I edit my responses?

Absolutely! Do not worry about filling it out perfectly in one go. If you create an account, you can edit your responses for the whole period the survey is open, and even fill it out across multiple devices, e.g. start on your phone, then fill out some on your desktop, etc. Even if you're filling it out anonymously, you can still edit responses on your device for some time, so you can have it open in a browser tab and revisit it periodically.

Why are there JS questions in an HTML survey?

This question comes up a lot every year.

For the same reason there are JS APIs in the HTML standard : many JS APIs are intrinsically related to HTML. We mainly included JS APIs that are in some way related to HTML, such as:

APIs used to manipulate HTML dynamically (DOM, interactivity, etc.)

Web Components APIs, used to create custom HTML elements

PWA features, including APIs used to access underlying system capabilities (OS capabilities, device capabilities, etc.)

The only two exceptions to this are two Intl APIs, which were mainly included because we wanted to get participants thinking about any localization/internationalization pain points they may have.

However, if you don't write any JS, we absolutely still want to hear from you! In fact, I would encourage you even more strongly to fill out the survey, as people who don't write JS are very underrepresented in these surveys. All questions are optional , so you can just skip any JS-related questions.

There is also a question at the end, where you can select that you only write HTML/CSS:

Call for Testing: Build Dir Layout v2

Ed Page · Rust Blog

We would welcome people to try and report issues with the nightly-only cargo -Zbuild-dir-new-layout . While the layout of the build dir is internal-only, many projects need to rely on the unspecified details due to missing features within Cargo. While we've performed a crater run , that won't cover everything and we need help identifying tools and process that rely on the details, reporting issues to these projects so they can update to the new layout or support them both.

How to test this?

With at least nightly 2026-03-10, run your tests, release processes, and anything else that may touch build-dir/target-dir with the -Zbuild-dir-new-layout flag.

For example:

```
$ cargo test -Zbuild-dir-new-layout
```

Note: if you see failures, the problem may not be isolated to just -Zbuild-dir-new-layout . With Cargo 1.91 , users can separate where to store intermediate build artifacts (build-dir) and final artifacts (still in target-dir). You can verify this by running with only CARGO_BUILD_BUILD_DIR=build set. We are evaluating changing the default for build-dir in #16147 .

Outcomes may include:

Fixing local problems

Reporting problems in upstream tools with a note on the the tracking issue for others

Providing feedback on the the tracking issue

Known failure modes:

Inferring a [[bin]] s path from a [[test]] s path:

Use std::env::var_os("CARGO_BIN_EXE_*") for Cargo 1.94+, maybe keeping the inference as a fallback for older Cargo versions

Use env!("CARGO_BIN_EXE_*")

Build scripts looking up target-dir from their binary or OUT_DIR : see Issue #13663

Update current workarounds to support the new layout

Looking up user-requested artifacts from rustc, see Issue #13672

Update current workarounds to support the new layout

Library support status as of publish time:

assert_cmd : fixed

cli_test_dir : Issue #65

compiletest_rs : Issue #309

executable-path : fixed

snapbox : fixed

term-transcript : Issue #269

test_bin : Issue #13

trycmd : fixed

What is not changing?

The layout of final artifacts within target dir .

Nesting of build artifacts under the profile and the target tuple, if specified.

What is changing?

We are switching from organizing by content type to scoping the content by the package name and a hash of the build unit and its inputs.

Here is an example of the current layout, assuming you have a package named lib and a package named bin , and both have a build script:

build-dir/

├─ CACHEDIR.TAG

└─ debug/

├─ .cargo-lock # file lock protecting access to this location

├─ .fingerprint/ # build cache tracking

| └─ bin-[BUILD_SCRIPT_RUN_HASH]/*

| └─ bin-[BUILD_SCRIPT_BIN_HASH]/*

| └─ bin-[HASH]/*

| └─ lib-[BUILD_SCRIPT_RUN_HASH]/*

| └─ lib-[BUILD_SCRIPT_BIN_HASH]/*

| └─ lib-[HASH]/*

└─ build/

| └─ bin-[BIN_HASH]/* # build script binary

| └─ bin-[RUN_HASH]/out/ # build script run OUT_DIR

| └─ bin-[RUN_HASH]/* # build script run cache

| └─ lib-[BIN_HASH]/* # build script binary

Copying object properties, the robust way

Lea Verou

If, like me, you try to avoid using heavy libraries when not needed, you must have definitely written a helper to copy properties from one object to another at some point. It's needed so often that it's just silly to write the same loops over and over again.

These days, most of my time is spent working on my research project at MIT, which I will hopefully reveal later this year. In that, I'm using a lightweight homegrown helper library, which I might release separately at some point as I think it has potential in its own right, for a number of reasons.

Of course, it needed to have a simple `extend()` method as well, to copy properties from one object to another. Let's assume for the purposes of this article that we're talking about shallow copying, that overwrites are allowed, and let's omit `hasOwnProperty()` checks to make code easier to read.

It's a simple task, right? Our first attempt might look like this:

```
$.extend = function (to, from) { for (var property in from)
{ to[property] = from[property]; }

return to; }
```

This works fine, until you try it on objects with accessors or other types of properties defined via `Object.defineProperty()` or `get / set` keywords. What do you do then? Our next iteration could look like this:

```
$.extend = function (to, from) { for (var property in from) { Object.defineProperty(to, property,
Object.getOwnPropertyDescriptor(from, property)); }

return to; }
```

This works much better, until it fails, and it can fail pretty epically. Try this:

```
$.extend(document.body.style, { backgroundColor: "red" });
```

Both in Chrome and Firefox, the results are super weird. Even though `document.body.style.backgroundColor` will return "red", no style will have actually been applied. In Firefox it even destroyed the native setter entirely and any future attempts to set `document.body.style.backgroundColor` in the console did absolutely nothing.

In contrast, the previous naïve approach worked fine for this. It's clear that we need to somehow combine the two approaches, using `Object.defineProperty()` only when actually needed. But when is it actually not needed?

One obvious case is if the descriptor is undefined (such as with some native properties). Also, in simple properties, such as those in our object literal, the descriptor will be of the form `{value: somevalue, writable: true, enumerable: true, configurable: true}`. So, the next obvious step would be:

```
$.extend = function (to, from) { var descriptor =
Object.getOwnPropertyDescriptor(from, property);
```

```
if (descriptor && (!descriptor.writable || !
descriptor.configurable || !descriptor.enumerable ||
descriptor.get || descriptor.set)) { Object.defineProperty(to,
property, descriptor); } else { to[property] = from[property]; } }
```

This works perfectly, but is a little clumsy. I've currently left it at that, but any suggestions for making it more elegant are welcome :)

FWIW, I looked at jQuery's implementation of `jQuery.extend()` after this, and it seems it doesn't even handle accessors at all, unless I missed something. Time for a pull request, perhaps...

Edit: As MaxArt pointed out in the comments, there is a similar native method in ES6, `Object.assign()`. However, it does not deal with copying accessors, so does not deal with this problem either.

The Cicada Principle, revisited with CSS variables

Lea Verou

So what is the Cicada Principle and how does it relate to web design in a nutshell? It boils down to: when using repeating elements (tiled backgrounds, different effects on multiple elements etc), using prime numbers for the size of the repeating unit maximizes the appearance of organic randomness. Note that this only works when the parameters you set are independent.

When I recently redesigned my blog, I ended up using a variation of the Cicada principle to pseudo-randomize the angles of code snippets. I didn't think much of it until I saw this tweet:

<https://twitter.com/StuRobson/status/1273904521132072960>

This made me think: hey, maybe I should actually write a blog post about the technique. After all, the technique itself is useful for way more than angles on code snippets.

The main idea is simple: You write your main rule using CSS variables, and then use `:nth-of-*()` rules to set these variables to something different every N items. If you use enough variables, and choose your Ns for them to be prime numbers, you reach a good appearance of pseudo-randomness with relatively small Ns.

In the case of code samples, I only have two different top cuts (going up or going down) and two different bottom cuts (same), which produce $2 \times 2 = 4$ different shapes. Since I only had four shapes, I wanted to maximize the pseudo-randomness of their order. A first attempt looks like this:

```
pre { clip-path: polygon(var(--clip-top), var(--clip-bottom));
clip-top: 0 0, 100% 2em; clip-bottom: 100% calc(100% - 1.5em), 0 100%; }
```

```
pre:nth-of-type(odd) { clip-top: 0 2em, 100% 0; }
```

```
pre:nth-of-type(3n + 1) { clip-bottom: 100% 100%,
calc(100% - 1.5em); }
```

This way, the exact sequence of shapes repeats every $2 \times 3 = 6$ code snippets. Also, the alternative `clip-bottom` doesn't really get the same visibility as the others, being present only 33.333% of the time. However, if we just add one more selector:

```
pre { clip-path: polygon(var(--clip-top), var(--clip-bottom));
clip-top: 0 0, 100% 2em; clip-bottom: 100% calc(100% - 1.5em), 0 100%; }
```

```
pre:nth-of-type(odd) { clip-top: 0 2em, 100% 0; }
```

```
pre:nth-of-type(3n + 1), pre:nth-of-type(5n + 1) { clip-bottom: 100% 100%, 0 calc(100% - 1.5em); }
```

Now the exact same sequence of shapes repeats every $2 \times 3 \times 5 = 30$ code snippets, probably way more than I will have in

any article. And it's more fair to the alternate `clip-bottom`, which now gets $1/3 + 1/5 - 1/15 = 46.67\%$, which is almost as much as the alternate `clip-top` gets!

You can explore this effect in this codepen:

<https://codepen.io/leaverou/pen/8541bfd3a42551f8845d668f29596ef9?editors=1100>

Or, to better explore how different CSS creates different pseudo-randomness, you can use this content-less version with three variations:

<https://codepen.io/leaverou/pen/NWxaPVx>

Of course, the illusion of randomness is much better with more shapes, e.g. if we introduce a third type of edge we get $3 \times 3 = 9$ possible shapes:

<https://codepen.io/leaverou/pen/dyGmbJJ?editors=1100>

I also used primes 7 and 11, so that the sequence repeats every 77 items. In general, the larger primes you use, the better the illusion of randomness, but you need to include more selectors, which can get tedious.

Other examples

So this got me thinking: What else would this technique be cool on? Especially if we include more values as well, we can pseudo-randomize the result itself better, and not just the order of only 4 different results.

So I did a few experiments.

Pseudo-randomized color swatches

<https://codepen.io/leaverou/pen/NWxXQKX>

Pseudo-randomized color swatches, with variables for hue, saturation, and lightness.

And an alternative version:

<https://codepen.io/leaverou/pen/RwrLPer>

Which one looks more random? Why do you think that is?

Pseudo-randomized border-radius

Admittedly, this one can be done with just longhands, but since I realized this after I had already made it, I figured eh, I may as well include it ☺

<https://codepen.io/leaverou/pen/ZEQXOrd>

It is also really cool when combined with pseudo-random colors (just hue this time):

<https://codepen.io/leaverou/pen/oNbGzeE>

Pseudo-randomized snowfall

CSS3 for developers: My Fronteers 2011 workshop

Lea Verou

In case you haven't noticed, in addition to my talk at Fronteers 2011, I'll also be holding a full day workshop the day before the conference. The title of that workshop is "CSS3 for developers" and I wanted to explain a bit what it's going to be about and why I chose to target web developers only.

Why "for developers"?

First of all, even though I do design websites and I really love design (not only web design), I consider myself primarily a developer. So, I think I can communicate better with other devs, rather than designers, since we "speak the same language". Secondly, most CSS3 talks and workshops are presented by and targeted to, designers. Developers end up feeling left out and in return they tend to consider CSS an inferior technology which isn't for them. CSS might not be a programming language, but it is code, and to fully master, it requires a very similar skillset to programming. It's no wonder that most people that actually do research on CSS and/or write the specifications are not designers.

Besides, CSS3, in essence, is about creating web applications that download faster and are easier to develop, maintain and edit. There are very few things that can't be done at all with CSS2.1. CSS3 just allows us to do them better: Less HTTP requests, less kilobytes to download, less presentational JavaScript, more flexibility. CSS3 is mostly about coding speed, flexibility, performance, maintainability. None of these are artistic pursuits, they're all purely developer goals!

What will it be about?

It will be about many well-implemented and popular CSS3 features, like border-radius, shadows, gradients, new background properties, selectors, media queries, transforms, transitions etc. The key difference from most CSS3 talks & workshops will be the depth these will be covered in and the different perspective (practical information rather than artistic or "inspirational"). I usually opt in for depth rather than breadth for my talks, and expect the same from this workshop. The feedback I get most frequently for my talks is "I thought I knew everything about topic X, but yet I learned so much!". I'll do my best to maintain this reputation for this workshop as well. ;)

In addition to learning how CSS3 stuff can be used, information about browser support, fallbacks and performance will be provided.

Watch a single person talk for a whole day? Booooooring!

Hey, I agree. And it's not just that: Listening to someone talk about a given topic and trying it out yourself are two very different things. I believe that you only really learn something when you actually use it. That's why it won't be done like that. ;) It will be very hands on and there is going to be at least one small exercise per almost everything explained.

The exercises are going to be performed in a little web app designed exclusively for this workshop, so that you won't need to bother with prefixes or write HTML and irrelevant CSS code just to try out a new property. And what's best, you can take that app at home with you and practice what you learned as much as you want!

Of course that means that every attendee will have to bring their own laptop (or borrow one from a friend).

Kewl, can I haz ticket?

Workshop attendance is priced at €350 (€275 for Fronteers members) and you can get your ticket here: <http://fronteers.paydro.net/>

To make the experience better and more educational, we limited the number of attendees to 30. That might mean you need to rush: there are currently tickets available, but in a few days there might not be any left!

Looking forward to seeing you in Amsterdam! :)

That “job brief” on Google Forms could infect your device

Malwarebytes Labs

We’ve identified a campaign using business-related lures, such as job interviews, project briefs, and financial document, to distribute malware, including the PureHVNC Remote Access Trojan (RAT).

It’s not the malware that’s new, but how the attack starts.

Instead of the usual phishing email or fake download page, attackers are using Google Forms to kick off the infection chain. The attack typically begins when a victim downloads a business-themed ZIP file linked from a Google Form. Inside is a malicious file that sets off a multi-stage infection process, eventually installing malware on the system.

What is PureHVNC?

PureHVNC is a modular .NET RAT from the “Pure” malware family. In simple terms, it gives attackers remote control over an infected device and lets them steal sensitive information.

Once installed, it can:

Take control of the system and run commands remotely.

Collect information about the device, including operating system, hardware, security software, and info about the user and connected devices.

Steal data from browsers, extensions and crypto wallets.

Extract data from apps like Telegram and Foxmail.

Install additional plugins.

Achieve persistence in several ways (for example, via scheduled tasks).

Different lures, same goal: compromise your device

In our research, we found multiple Google Forms hosting links to malicious ZIP files that start the infection chain. These forms are convincing, impersonating real company names, logos and links. LinkedIn is one of the platforms used to send links to these malicious forms.

Fake Google Forms that distribute malicious ZIPs.

The attackers impersonate real companies

Well-known brands are impersonated to lend credibility

The forms typically ask for professional information (experience, background, etc.), making them feel like part of a real recruitment or business process.

Information requested from the user to make the form appear legitimate.

More information.

The forms link to ZIP files hosted on:

File-sharing services such as Dropbox, filedn.com, and fshare.vn

URL shorteners such as tr.ee and goo.su

Google redirect links that obscure the final destination

The ZIP archives use various names and are tied to different business-related themes (marketing, interviews, projects, job offers, budgets, partnerships, benefits) to avoid suspicion, for example:

{CompanyName}_GlobalLogistics_Ad_Strategy.zip

Project_Information_Summary_2026.zip

{CompanyName} Project 2026 Interview Materials.zip

{CompanyName}_Company_and_Job_Overview.pdf.rar

Collaboration Project with {CompanyName} Company 2026.zip

The lures use the names of well-known companies, particularly in the financial, logistic, technology, sustainability and energy sectors. Impersonating legitimate organizations add credibility to their campaign.

What happens after you download the file

The ZIP archives usually contain legitimate files (such as PDFs of job descriptions) and an executable file along with a DLL, typically named msimg32.dll. The DLL is executed via DLL hijacking (tricking a legitimate program into loading malicious code), although the technique has undergone multiple modifications and upgrades over time.

Legitimate PDFs are present in some ZIP files, like this one masquerading as a real job description.

Analysis of the malicious campaign

We identified multiple variants of this campaign, each using different methods to extract the archive, distinct Python code, and varying folder structures. Across these variants, the campaign typically includes an executable file along with a DLL hidden in a separate folder. In some cases, attackers also include legitimate files related to the lure’s theme, enhancing the overall credibility of the attack.

Example of files present in one of the archives analyzed.

The malicious code is present in the DLL, and carries out various operations, including:

Decrypting strings with a simple XOR, in this case with the “4B” key.

Refresh CSS Bookmarklet v2

Lea Verou

Almost 11 years ago, Paul Irish posted this brilliant bookmarklet to refresh all stylesheets on the current page. Despite the amount of tools, plugins, servers to live reload that have been released over the years, I've always kept coming back to it. It's incredibly elegant in its simplicity. It works everywhere: locally or remotely, on any domain and protocol. No need to set up anything, no need to alter my process in any way, no need to use a specific local server or tool. It quietly just accepts your preferences and workflow instead of trying to change them. Sure, it doesn't automatically detect changes and reload, but in most cases, I don't want it to.

I've been using this almost daily for a decade and there's always been one thing that bothered me: It doesn't work with iframes. If the stylesheet you're editing is inside an iframe, tough luck. If you can open the frame in a new tab, that works, but often that's nontrivial (e.g. the frame is dynamically generated). After dealing with this issue today once more, I thought "this is just a few lines of JS, why not fix it?".

The first step was to get Paul's code in a readable format, since the bookmarklet is heavily minified:

```
(function() { var links = document.getElementsByTagName("link");
for (var i = 0; i < links.length; i++) { for (let link
of Array.from(document.querySelectorAll("link[rel=stylesheet]
[href]"))) { var href = new
URL(link.href, location); href.searchParams.set("forceRe-
load", Date.now());
link.href = href; } } })()
```

Sure, this reduces browser support a bit (most notably it excludes IE11), but since this is a local development tool, that's not such a big problem.

Now, let's extend this to support iframes as well:

```
for (let iframe of
document.getElementsByTagName("iframe")) {
  if (iframe.contentDocument) {
    refresh(iframe.contentDocument);
  }
}
```

That's it! Do keep in mind that this will not work with cross-origin iframes, but then again, you probably don't expect it to in that case.

Now all we need to do to turn it into a bookmarklet is to prepend it with javascript: and minify the code. Here you go:

☐ CSS v2

Hope this is useful to someone else as well :) Any improvements are always welcome!

Paul Irish, for the original bookmarklet

Maurício Kishi, for making the iframe traversal recursive (comment)

Security advisory for Cargo

The Rust Security Response Team · Rust Blog

The Rust Security Response Team was notified of a vulnerability in the third-party crate `tar`, used by Cargo to extract packages during a build. The vulnerability, tracked as CVE-2026-33056, allows a malicious crate to change the permissions on arbitrary directories on the filesystem when Cargo extracts it during a build.

For users of the public crates.io registry, we deployed a change on March 13th to prevent uploading crates exploiting this vulnerability, and we audited all crates ever published. We can confirm that no crates on crates.io are exploiting this.

For users of alternate registries, please contact the vendor of your registry to verify whether you are affected by this. The Rust team will release Rust 1.94.1 on March 26th, 2026, updating to a patched version of the `tar` crate (along with other non-security fixes for the Rust toolchain), but that won't protect users of older versions of Cargo using alternate registries.

We'd like to thank Sergei Zimmerman for discovering the underlying `tar` crate vulnerability and notifying the Rust project ahead of time, and William Woodruff for directly assisting the crates.io team with the mitigations. We'd also like to thank the Rust project members involved in this advisory: Eric Huss for patching Cargo; Tobias Bieniek, Adam Harvey and Walter Pearce for patching crates.io and analyzing existing crates; Emily Albini

and Josh Stone for coordinating the response; and Emily Albini for writing this advisory.

For users of alternate registries, please contact the vendor of your registry to verify whether you are affected by this.

The Rust Security Response Team

BRIEFS

IN BRIEF

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): Now that Node.js has built-in support for TypeScript, we can use it as the foundation of simple playgrounds that let us interactively explore TypeScript code.

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): In the last chapter, we worked with numbers. In this chapter, we'll work with text and write our first applications.

Lea Verou, Lea Verou: Today I needed to quickly find the Gravatars that corresponded to a bunch of email addresses for some reason (don't ask). After a bit of googling and wandering around in Gravatar's official site and others, I figured out it's probably much quicker to write a short PHP script for that myself, than keep looking.

Here it is, in case someone ever needs to do something similar: (click on the screenshot)

(has anyone noticed my latest love affair with Helvetica/Arial? : P)

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): In JavaScript, code has color: It is either synchronous or asynchronous. In this blog post, we explore:

The problems caused by that

How to fix them via synchronous await

The two downsides that prevent synchronous await from being practical

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): JavaScript's Arrays are so flexible that TypeScript provides two different kinds of types for handling them:

Array types for arbitrary-length sequences of values that all have the same type – e.g.: Array

Tuple types for fixed-length sequences of values where each one may have a different type – e.g.: [number, string, boolean]

In this blog post, we look at the latter – especially how to compute with tuples at the type level.

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): TypeScript's satisfies operator lets us check the type of a value (mostly) without influencing it. In this blog post, we examine how exactly it works and where it's useful.

PortSwigger Web Security Blog, PortSwigger Web Security Blog: 1000s of pentesters are currently using Burp AI features to hack smarter by eliminating tedious tasks and delivering instant insights, right inside Burp Suite. Security Researcher John Hammond took Bu

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): Node.js is a program that lets us run JavaScript code outside browsers – which we can use for a variety of things.

Aaron Nyzil D'souza, Haptik Tech:

We live in a world where communicating or talking to machines has become super normal. You ask your phone for the weather forecast, chat with a bot to know your food delivery status or even command Alexa to play one of your favorite songs — and boom, it just works.

The Courier

Vol. 1, No. 072 · 2026-03-30

Curated and composed automatically.